



دانشگاه صنعتی شریف
دانشکده مهندسی کامپیوتر

عنوان:

گزارش نهایی پروژه اینترنت اشیا

طراحی و پیاده‌سازی سامانه مدیریت OTA برای ESP32

**IOT Project Final Report - ESP32 OTA
Management System**

اعضای گروه

علی فتحی

آرمین فارسی

امیررضا بهرامی

نام درس

اینترنت اشیا

تابستان ۱۴۰۵

نام استاد درس

دکتر اجلالی

چکیده

هدف این پروژه طراحی و پیاده‌سازی یک سامانه کامل برای مدیریت به‌روزرسانی از راه دور ثابت‌افزار روی بورد ESP32 بود. سامانه نهایی از چهار بخش اصلی شامل کلاینت ESP32، سرور FastAPI، پایگاه داده SQLite و پنل مدیریتی وب تشکیل شده است. در مسیر توسعه، ابتدا قراردادهای ارتباطی و ساختار پایگاه داده طراحی شدند، سپس فرآیند دریافت و نصب واقعی ثابت‌افزار با استفاده از دو پارتیشن OTA پیاده‌سازی شد. در ادامه، بررسی صحت فایل با SHA-256، نگهداری وضعیت عملیات در NVS، اعتبارسنجی نسخه جدید و سازوکار Rollback به سامانه اضافه شدند.

در مرحله نهایی، حالت‌های به‌روزرسانی دستی و خودکار، Heartbeat، بررسی دوره‌ای نسخه جدید، مدیریت فعال و غیرفعال کردن Firmware، جلوگیری از ایجاد عملیات تکراری و ثبت Idempotent وضعیت‌ها تکمیل شدند. همچنین مشکلات اجرایی مشاهده‌شده در تست‌ها، از جمله Stack Overflow در سرویس دوره‌ای، تکرار نسخه ناموفق پس از Rollback و ناسازگاری وضعیت‌های باقی‌مانده در NVS برطرف شدند. در پایان، مجموعه تست‌های نهایی شامل OTA موفق، جابه‌جایی بین دو پارتیشن، Rollback، حالت خودکار و دستی، قطع و بازیابی شبکه، خطای SHA-256 و بازیابی ارتباط با سرور با موفقیت انجام شدند.

۱. مقدمه و هدف پروژه

در سامانه‌های اینترنت اشیا، امکان به‌روزرسانی ثابت‌افزار بدون دسترسی فیزیکی به دستگاه یکی از نیازهای اصلی نگهداری و توسعه است. هدف این پروژه ایجاد سامانه‌ای بود که بتواند نسخه‌های ثابت‌افزار را در سرور مدیریت کند، دستگاه‌های ESP32 را شناسایی و نسخه فعلی آن‌ها را ثبت نماید، فرآیند به‌روزرسانی را به صورت دستی یا خودکار آغاز کند و نتیجه هر عملیات را به شکل قابل پیگیری در سرور نگهداری نماید.

توسعه پروژه طی چهار هفته انجام شد. در هفته اول زیرساخت ارتباطی، پایگاه داده، سرور و پنل آماده شدند. در هفته دوم نصب واقعی ثابت‌افزار روی پارتیشن‌های OTA پیاده‌سازی شد. هفته سوم به بررسی صحت فایل با SHA-256 و پیاده‌سازی Rollback اختصاص یافت و در هفته چهارم حالت خودکار، سرویس‌های دوره‌ای، مدیریت چرخه Firmware و مقاوم‌سازی بخش‌های اجرایی تکمیل شدند.

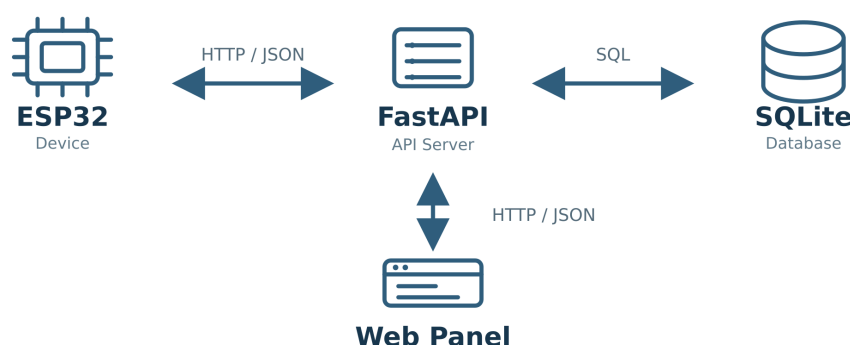
جدول ۱: خلاصه روند توسعه پروژه

مرحله	خروجی اصلی
هفته اول	طراحی API و پایگاه داده، راه‌اندازی بورد، سرور و پنل مدیریتی
هفته دوم	طراحی پارتیشن‌های OTA، دانلود و نصب واقعی نسخه جدید و ثبت وضعیت عملیات
هفته سوم	بررسی SHA-256، اعتبارسنجی نسخه جدید، Rollback و جلوگیری از حلقه به‌روزرسانی ناموفق
هفته چهارم	Heartbeat، بررسی دوره‌ای، Automatic Update، مدیریت Firmware و مقاوم‌سازی سرویس‌ها

۲. معماری کلی سامانه

معماری نهایی از چهار جزء تشکیل شده است: بورد ESP32 به عنوان کلاینت، سرور FastAPI برای ارائه APIها و مدیریت منطق سامانه، پایگاه داده SQLite برای ذخیره وضعیت دستگاهها و عملیاتها، و پنل مدیریتی مبتنی بر HTML/CSS/JavaScript.

ارتباط میان بورد و سرور با HTTP/JSON انجام می‌شود. پنل نیز از همان APIهای سرور استفاده می‌کند و سرور از طریق SQL به پایگاه داده متصل است.



شکل ۱: معماری کلی سامانه شامل ESP32، سرور FastAPI، پایگاه داده SQLite و پنل مدیریتی

۲-۱. قرارداد ارتباط بورد و سرور

دو مسیر پایه از ابتدای پروژه برای ارتباط دستگاه با سرور تعریف شدند:

روش	مسیر	کاربرد
POST	/api/devices/register	ثبت دستگاه و به‌روزرسانی Heartbeat
GET	/api/devices/{uid}/update	بررسی وجود عملیات یا نسخه جدید

شناسه یکتای دستگاه از آدرس MAC بورد ساخته می‌شود و نسخه برنامه از اطلاعات خود ESP-IDF خوانده می‌شود. به این ترتیب سرور می‌تواند دستگاه را بدون تعریف دستی شناسایی کرده و نسخه واقعی در حال اجرا را نگهداری کند.

۲-۲. ساختار پایگاه داده

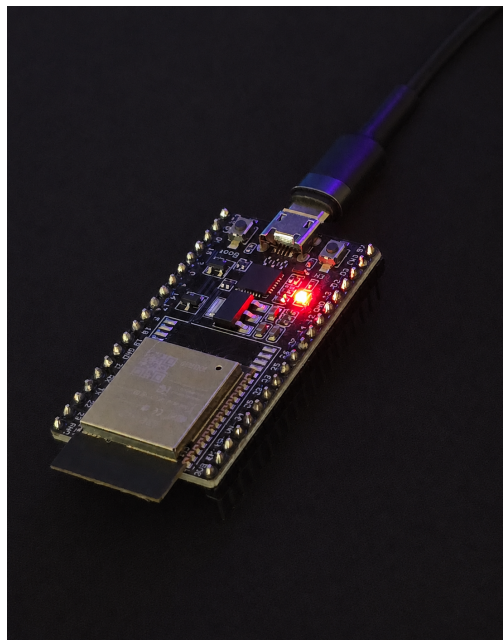
پایگاه داده از چهار جدول اصلی تشکیل شده است:

- **devices**: شناسه دستگاه، نسخه فعلی، حالت manual/automatic، آخرین وضعیت و زمان آخرین ارتباط.
- **firmwares**: نسخه ثابت‌افزار، نام و مسیر فایل، اندازه، SHA-256، توضیحات و وضعیت فعال یا غیرفعال.
- **update_jobs**: هر عملیات به‌روزرسانی شامل دستگاه مقصد، نسخه مقصد، نوع شروع عملیات، وضعیت و زمان‌های شروع و پایان.
- **update_events**: تاریخچه مرحله‌به‌مرحله وضعیت‌های هر عملیات.

این تفکیک باعث شد وضعیت جاری دستگاه از تاریخچه عملیات مستقل باشد و در عین حال تمام تغییرات قابل ردیابی باقی بمانند.

۳. راه‌اندازی و ساختار کلاینت ESP32

بورد مورد استفاده ESP32 DevKitC V4 است و توسعه با ESP-IDF انجام شد. برای فرآیند توسعه نیز اسکریپت‌های جداگانه برای آماده‌سازی محیط، Build/Flash، مشاهده خروجی سریال و پاک‌سازی خروجی ایجاد شدند.



شکل ۲: بورد ESP32 DevKitC V4 مورد استفاده در پروژه

نسخه ثابت‌افزار به کمک متغیر نسخه پروژه در CMake تعیین می‌شود و در زمان اجرا با `esp_app_get_description()` خوانده می‌شود. شناسه دستگاه نیز با خواندن آدرس MAC تولید می‌گردد.

۴. طراحی حافظه و پارتیشن‌های OTA

در ابتدای توسعه واقعی OTA مشخص شد حافظه فیزیکی مورد نیاز برابر 4MB است، در حالی که تنظیم اولیه پروژه روی 2MB قرار داشت. پس از اصلاح این تنظیم، یک جدول پارتیشن اختصاصی طراحی شد که امکان نگهداری نسخه فعلی و نسخه جدید را به صورت همزمان فراهم می‌کند.

#	Name,	Type,	SubType,	Offset,	Size
1	nvs,	data,	nvs,	0x9000,	0x4000
2	otadata,	data,	ota,	0xd000,	0x2000
3	phy_init,	data,	phy,	0xf000,	0x1000
4	factory,	app,	factory,	0x10000,	0x140000
5	ota_0,	app,	ota_0,	0x150000,	0x150000
6	ota_1,	app,	ota_1,	0x2A0000,	0x150000

پارتیشن factory نسخه اولیه را نگهداری می‌کند، دو پارتیشن ota_0 و ota_1 به صورت متناوب برای نسخه‌های جدید استفاده می‌شوند و otadata اطلاعات لازم برای انتخاب پارتیشن بوت را نگهداری می‌کند.

```
Parsing binary partition input...
Verifying table...
# ESP-IDF Partition Table
# Name, Type, SubType, Offset, Size, Flags
nvs,data,nvs,0x9000,16K,
otadata,data,ota,0xd000,8K,
phy_init,data,phy,0xf000,4K,
factory,app,factory,0x10000,1280K,
ota_0,app,ota_0,0x150000,1344K,
ota_1,app,ota_1,0x2a0000,1344K,
```

شکل ۳: نمونه خروجی جدول پارتیشن شامل factory, otadata, ota_0 و ota_1

۵. دریافت و نصب ثابت افزار

پس از دریافت پاسخ بررسی نسخه، اطلاعات عملیات در یک ساختار شامل شناسه عملیات، نسخه مقصد، شناسه Firmware، آدرس فایل، اندازه و مقدار SHA-256 نگهداری می‌شوند. در صورت وجود نسخه جدید، بورد فایل را به صورت مستقیم از سرور دریافت می‌کند.

برای نصب، ابتدا پارتیشن مقصد با `esp_ota_get_next_update_partition()` تعیین می‌شود. سپس با `esp_ota_begin()` عملیات نوشتن آغاز شده و داده‌ها در بلوک‌های 4096 بایتی با `esp_ota_write()` روی پارتیشن مقصد قرار می‌گیرند. پس از پایان، `esp_ota_end()` صحت ساختار تصویر را بررسی می‌کند و `esp_ota_set_boot_partition()` پارتیشن بوت بعدی را مشخص می‌نماید.

```
I (6773) main: Update available!
I (6773) main: Target version: 1.1.0
I (6773) main: Firmware ID: 1
I (6783) main: Firmware size: 892336 bytes
I (6783) main: Firmware URL: http://192.168.45.216:8000/api/firmwares/1/download
I (6793) main: SHA-256: 7d354ed0b2cc6df060ed9af779c3ee9428a33789d611d292b5aa749d8efa6fb7
I (6803) api_client: Reporting OTA status: downloading
I (6853) api_client: Response status: 200
I (6853) api_client: Response body: {"update_id":3,"status":"downloading"}
I (6863) main: Starting OTA update to version 1.1.0...
I (6863) ota_manager: Starting OTA update
I (6863) ota_manager: Firmware URL: http://192.168.45.216:8000/api/firmwares/1/download
I (6873) ota_manager: Target partition: ota_0
I (6913) ota_manager: Firmware size: 892336 bytes
I (9403) ota_manager: Written 4096 bytes
I (9573) ota_manager: Written 8192 bytes
I (9593) ota_manager: Written 12288 bytes
I (9613) ota_manager: Written 16384 bytes
I (9633) ota_manager: Written 20480 bytes
I (9663) ota_manager: Written 24576 bytes
I (9683) ota_manager: Written 28672 bytes
I (9713) ota_manager: Written 32768 bytes
I (9733) ota_manager: Written 36864 bytes
I (9763) ota_manager: Written 40960 bytes
I (9793) ota_manager: Written 45056 bytes
I (9823) ota_manager: Written 49152 bytes
I (9843) ota_manager: Written 53248 bytes
I (9863) ota_manager: Written 57344 bytes
I (9903) ota_manager: Written 61440 bytes
```

شکل ۴: نمونه دریافت نسخه جدید و نوشتن مرحله‌ای ثابت‌افزار روی پارتیشن مقصد

مزیت اصلی این ساختار آن است که نسخه قبلی هنگام نصب نسخه جدید حذف نمی‌شود؛ بنابراین در صورت شکست نسخه جدید، امکان بازگشت به نسخه سالم قبلی وجود دارد.

۶. بررسی صحت فایل با SHA-256

سرور در زمان بارگذاری هر فایل ثابت‌افزار، اندازه و مقدار SHA-256 آن را محاسبه و در پایگاه داده ذخیره می‌کند. این مقدار همراه با اطلاعات Update برای بورد ارسال می‌شود.

در سمت ESP32، محاسبه هش به صورت جریانی و هم‌زمان با دانلود انجام می‌شود. هر بلوک دریافت‌شده ابتدا وارد محاسبه SHA-256 شده و سپس روی پارتیشن نوشته می‌شود. پس از پایان دانلود، مقدار نهایی با هش دریافت‌شده از سرور مقایسه می‌شود.

```
1 mbedtls_md_update(&sha_ctx, buffer, read_len);
2
3 err = esp_ota_write(
4     ota_handle,
5     buffer,
6     read_len
7 );
```

در صورت عدم تطابق دو مقدار، عملیات با `esp_ota_abort()` متوقف شده و خطای `ESP_ERR_INVALID_CRC` بازگردانده می‌شود. در نتیجه تصویر مخدوش یا متفاوت از نسخه ثبت‌شده در سرور، به عنوان نسخه معتبر بوت نمی‌شود.

```
I (21353) ota_manager: Calculated SHA-256: f48d7324be922ed99c78a4af34883d405bc2b426a3f0aa5a887b03ebacf601e3
I (21353) ota_manager: Expected SHA-256: f48d7324be922ed99c78a4af34883d405bc2b426a3f0aa5a887b03ebacf601e3
I (21363) ota_manager: SHA-256 verification successful
```

شکل ۵: تطابق مقدار محاسبه‌شده و مقدار مورد انتظار SHA-256

۷. نگهداری وضعیت عملیات در NVS

از آن‌جا که نصب OTA با راه‌اندازی مجدد دستگاه همراه است، بخشی از وضعیت عملیات باید از یک اجرای برنامه به اجرای بعدی منتقل شود. برای این منظور از حافظه NVS استفاده شد.

اطلاعاتی مانند شناسه عملیات و نسخه مقصد پیش از راه‌اندازی مجدد ذخیره می‌شوند. در نسخه نهایی، وضعیت مرحله‌ای عملیات نیز برای تشخیص دقیق‌تر مسیرهای موفق، شکست و Rollback در NVS نگهداری شد.

وضعیت‌های مورد استفاده در طول عملیات شامل `installing`, `verified`, `downloading`, `pending`, `validating` و `success` هستند. در مسیرهای خطا نیز نتیجه نهایی با وضعیت `failed` یا `rolled_back` ثبت می‌شود.

پاک کردن وضعیت موقت تنها زمانی انجام می‌شود که نتیجه نهایی با موفقیت به سرور گزارش شده باشد. این رفتار امکان بازیابی و ارسال مجدد گزارش نهایی پس از قطع موقت شبکه یا سرور را فراهم می‌کند.

۸. اعتبارسنجی نسخه جدید و Rollback

قابلیت Rollback با فعال‌سازی گزینه زیر در ESP-IDF فراهم شد:

```
1 CONFIG_BOOTLOADER_APP_ROLLBACK_ENABLE=y
```

پس از نصب نسخه جدید، تصویر برای نخستین بار در وضعیت ESP_OTA_IMG_PENDING_VERIFY اجرا می‌شود. در این مرحله برنامه آزمون اعتبارسنجی اولیه را اجرا می‌کند.

اگر آزمون موفق باشد، تصویر جدید با `esp_ota_mark_app_valid_cancel_rollback()` معتبر اعلام شده و سپس وضعیت SUCCESS به سرور گزارش می‌شود.

در مسیر شکست، تصویر جدید نامعتبر اعلام شده و تابع `esp_ota_mark_app_invalid_rollback_and_reboot()` باعث بازگشت بوت‌لودر به نسخه سالم قبلی می‌شود.

```
W (620) main: Firmware verification failed. Rollback required
W (620) main: Rollback detected
```

شکل ۶: تشخیص شکست اعتبارسنجی و آغاز فرآیند Rollback

یکی از مشکلات مشاهده‌شده در تست‌های اولیه این بود که پس از بازگشت، همان نسخه ناموفق دوباره برای دستگاه پیشنهاد می‌شد و چرخه تکرار به‌وجود می‌آمد. با ثبت نتیجه نهایی، بستن Job قبلی و جلوگیری از پیشنهاد مجدد نسخه‌های failed/rolled_back به همان دستگاه، این حلقه حذف شد.

۹. سرور FastAPI و مدیریت عملیات

سرور با Python/FastAPI پیاده‌سازی شده و هنگام راه‌اندازی پایگاه داده را آماده می‌کند. علاوه بر مسیرهای مورد استفاده بورد، مسیرهای لازم برای مدیریت Firmware، ایجاد Update دستی، دریافت تاریخچه و تغییر حالت دستگاه نیز در اختیار پنل قرار دارند.

هر Update به عنوان یک رکورد مستقل در `update_jobs` ثبت می‌شود و تغییر وضعیت‌ها در `update_events` نگهداری می‌گردد. در نسخه نهایی، منطق سرور به گونه‌ای مقاوم‌سازی شد که:

- برای هر دستگاه در یک لحظه فقط یک عملیات باز وجود داشته باشد.
- گزارش تکراری همان وضعیت به صورت Idempotent پذیرفته شود و Event تکراری نسازد.
- گزارش دیررس یک وضعیت قدیمی، وضعیت Job را به عقب بازنگرداند.
- وضعیت‌های نهایی success، failed و rolled_back پایدار باقی بمانند.
- نسخه‌های ناموفق یا rolled_back در حالت خودکار دوباره به همان دستگاه پیشنهاد نشوند.

۱۰. مدیریت Firmware

صفحه مدیریت ثابت‌افزار امکان بارگذاری نسخه جدید، مشاهده Metadata و کنترل فعال یا غیرفعال بودن نسخه را فراهم می‌کند.

هنگام بارگذاری، سرور موارد زیر را ثبت می‌کند:

- شماره نسخه
- نام و مسیر فایل
- اندازه فایل
- مقدار SHA-256
- توضیحات تغییرات
- وضعیت فعال یا غیرفعال

نسخه غیرفعال در حالت Automatic انتخاب نمی‌شود و ایجاد Update دستی با آن نیز رد می‌گردد. همچنین نسخه تکراری مجدداً در پایگاه داده ثبت نمی‌شود.

ID	VERSION	SIZE	SHA-256	CHANGELOG	ACTIVE	UPLOADED
13	1.55.0	893.1 KB	53fcef3dd...	OTA test release 1.55.0	Activate	9/8/2026, 10:45:50 AM
11	1.54.0	893.1 KB	c27f4e28836e...	OTA test release 1.54.0	Activate	9/7/2026, 7:00:11 PM
10	1.53.0	891.5 KB	a2e3b931c758...	OTA test release 1.53.0	Deactivate	9/7/2026, 6:47:43 PM
9	1.52.0	891.5 KB	b86247f7b48b...	OTA test release 1.52.0	Deactivate	9/7/2026, 12:00:55 PM
6	1.51.0	890.4 KB	dad629076dbf...	OTA test release 1.51.0	Deactivate	9/7/2026, 11:02:13 AM

شکل ۷: رابط مدیریت Firmware و کنترل وضعیت فعال یا غیرفعال

۱۱. حالت دستی و خودکار به‌روزرسانی

۱۱-۱. حالت دستی

در حالت manual، مدیر از طریق پنل دستگاه و نسخه مقصد را انتخاب می‌کند. یک Job با وضعیت اولیه pending ایجاد شده و مورد در درخواست بعدی بررسی نسخه، اطلاعات همان Job را دریافت می‌کند. این حالت برای کنترل مستقیم عملیات، تست نسخه‌ها و مدیریت دستگاه‌های خاص مناسب است.

۱۱-۲. حالت خودکار

در حالت **automatic**، دستگاه در بازه‌های زمانی مشخص وجود نسخه جدید را از سرور بررسی می‌کند. اگر **Job** باز وجود نداشته باشد، سرور از میان **Firmware** های فعال، جدیدترین نسخه‌ای را که از نسخه جاری دستگاه جدیدتر است انتخاب کرده و **Job** خودکار ایجاد می‌کند.

انتخاب نسخه بر اساس ترتیب معنایی نسخه‌ها انجام می‌شود و نسخه‌ای که برای همان دستگاه قبلاً با **failed** یا **rolled_back** خاتمه یافته باشد از انتخاب خودکار حذف می‌شود.

Registered devices					
Live data from GET /api/devices , refreshed every 10 seconds.					
ID	DEVICE UID	VERSION	MODE	LAST STATUS	LAST SEEN
1	● ESP32-1CC3ABB3AAB8	1.54.0	automatic ▾	success	9/8/2026, 11:37:16 AM

شکل ۸: نمونه تنظیم حالت Automatic Update در سامانه

۱۲. Heartbeat و بررسی دوره‌ای Update

در هفته چهارم، رفتار ثبت دستگاه به عنوان **Heartbeat** دوره‌ای نیز مورد استفاده قرار گرفت. در پیاده‌سازی نهایی، سرویس دستگاه به صورت دوره‌ای:

- هر ۳۰ ثانیه وضعیت حضور دستگاه را با مسیر **Register** به سرور اعلام می‌کند.
- هر ۶۰ ثانیه وجود نسخه جدید را بررسی می‌کند.

در صورت قطع **WiFi**، سرویس تلاش می‌کند ارتباط را بازیابی کند و پس از بازگشت شبکه، **Heartbeat** و بررسی **Update** ادامه پیدا می‌کنند.

۱۳. مقاوم‌سازی سرویس اجرایی ESP32

در نخستین نسخه **Periodic Update**، حلقه سرویس داخل **Task** اصلی برنامه اجرا می‌شد. هنگام آغاز **OTA** از داخل همین حلقه، عمق فراخوانی‌ها و مصرف **Stack** افزایش یافت و خطای **Stack Overflow** مشاهده شد. برای رفع مشکل، **Heartbeat** و **Periodic Update Check** به یک **Task** مستقل **FreeRTOS** منتقل شدند. همچنین:

- **Stack** مناسب برای **Task** سرویس در نظر گرفته شد.
- از شروع همزمان دو **OTA** جلوگیری شد.

- **Heartbeat و Update Check** هنگام اجرای OTA با عملیات نصب تداخل ندارند.

- برای اتصال WiFi زمان انتظار محدود در نظر گرفته شد.

- وضعیت نهایی گزارش نشده پس از بازیابی شبکه دوباره ارسال می‌شود.

- **Stack Watermark** سرویس برای بررسی سلامت اجرایی قابل ثبت است.

پس از این تغییر، اجرای دوره‌ای دستگاه و شروع OTA در زمان فعالیت سرویس بدون **Stack Overflow** ادامه پیدا کرد.

۱۴. تصمیم نهایی درباره پروتکل ارتباطی

در پایان گزارش هفته سوم، انتقال ارتباط از HTTP به MQTT به عنوان برنامه مرحله بعد مطرح شده بود. با این حال، در پیاده‌سازی نهایی HTTP/JSON باقی ماند.

۱۵. جریان‌های اصلی عملکرد سامانه

۱-۱۵ Update موفق

در این مسیر نسخه جدید روی پارتیشن مقصد نوشته می‌شود، پس از تطبیق هش به عنوان پارتیشن بوت انتخاب می‌گردد، در نخستین بوت اعتبارسنجی می‌شود و پس از تأیید، وضعیت نهایی در سرور ثبت می‌شود. پس از دریافت پاسخ موفق از سرور، وضعیت موقت مربوط به عملیات نیز از NVS پاک می‌شود.

۲-۱۵ Rollback

در سناریوی Rollback، نسخه جدید پس از نخستین بوت اعتبارسنجی می‌شود. اگر آزمون اعتبارسنجی ناموفق باشد، تصویر جدید نامعتبر اعلام شده و بوت‌لودر نسخه سالم قبلی را دوباره اجرا می‌کند. پس از بازگشت، وضعیت **rolled_back** برای سرور ارسال شده و پس از ثبت موفق نتیجه، وضعیت موقت عملیات پاک می‌شود.

۳-۱۵ بازیابی پس از خطای شبکه

اگر ارتباط هنگام گزارش یک وضعیت نهایی در دسترس نباشد، وضعیت در NVS باقی می‌ماند. سرویس دستگاه پس از برقراری مجدد ارتباط، نتیجه را دوباره به سرور گزارش می‌کند و پس از دریافت پاسخ موفق، وضعیت موقت پاک می‌شود.

۱۶. تست‌های نهایی

پس از تکمیل توسعه، مجموعه تست‌های نهایی روی سامانه انجام شد. همه موارد زیر با نتیجه موفق پایان یافتند.

جدول ۲: نتایج تست‌های نهایی سامانه

ردیف	سناریو	نتیجه مورد انتظار و مشاهده شده	وضعیت
۱	OTA موفق عادی	دانلود، تطبیق SHA-256، نصب، بوت نسخه جدید و ثبت success	موفق
۲	جابه‌جایی بین OTA slots	نصب متناوب روی ota_0 و ota_1 و بوت صحیح هر دو مسیر	موفق
۳	Rollback	شکست اعتبارسنجی، بازگشت به نسخه قبلی و ثبت rolled_back	موفق
۴	Automatic Update	تشخیص نسخه فعال جدید بدون ایجاد Job دستی و شروع خودکار OTA	موفق
۵	حالت Manual	عدم شروع خودکار و آغاز Update پس از ایجاد Job دستی	موفق
۶	Active/Inactive Firmware	عدم پیشنهاد نسخه غیرفعال و قابل استفاده شدن مجدد پس از فعال‌سازی	موفق
۷	Heartbeat/Periodic Check	ادامه سرویس دوره‌ای بدون Crash و بدون Stack Overflow	موفق
۸	قطع و بازیابی WiFi	عدم Crash، تلاش برای اتصال مجدد و ادامه سرویس پس از بازیابی شبکه	موفق
۹	خطای SHA-256	لغو OTA، ثبت failed و باقی ماندن نسخه قبلی	موفق
۱۰	قطع شبکه هنگام OTA	شکست کنترل‌شده عملیات و حفظ امکان ادامه کار و گزارش نتیجه	موفق
۱۱	قطع و بازیابی سرور	عدم Crash دستگاه و بازیابی گزارش‌ها پس از بازگشت سرور	موفق
۱۲	جلوگیری از Retry نسخه خراب	عدم پیشنهاد مجدد Firmware دارای failed/rolled_back در حالت خودکار	موفق

۱۷. مشکلات مهم مشاهده شده و راه‌حل‌ها

۱۷-۱. تنظیم نادرست اندازه Flash

تنظیم اولیه پروژه با ظرفیت واقعی بورد سازگار نبود. پس از تشخیص حافظه 4MB، جدول پارتیشن مناسب طراحی و ظرفیت پروژه اصلاح شد.

۲-۱۷. حلقه تکرار پس از Rollback

در تست‌های اولیه نسخه ناموفق پس از بازگشت دوباره به دستگاه پیشنهاد می‌شد. با بستن عملیات شکست‌خورده، نگهداری وضعیت مناسب و حذف نسخه failed/rolled_back از انتخاب خودکار، حلقه برطرف شد.

۳-۱۷. Periodic Update در Stack Overflow

اجرای Service Loop در Task اصلی در زمان شروع OTA باعث Overflow می‌شد. انتقال سرویس به Task مستقل FreeRTOS این مشکل را رفع کرد.

۴-۱۷. گزارش‌های تکراری و بازگشت وضعیت

به دلیل retry شبکه، یک وضعیت ممکن بود چند بار به سرور برسد یا یک گزارش قدیمی دیرتر از مرحله جدید دریافت شود. ثبت Idempotent و جلوگیری از حرکت وضعیت به عقب باعث شد تاریخچه عملیات سازگار باقی بماند.

۵-۱۷. وضعیت باقی‌مانده در NVS

در طول توسعه، فلش دستی نسخه‌ها می‌توانست باعث باقی ماندن State مربوط به OTA قبلی شود. در نسخه نهایی، پارتیشن در حال اجرا و وضعیت تصویر بررسی می‌شوند تا State قدیمی با بوت واقعی OTA اشتباه گرفته نشود.

۱۸. وضعیت نهایی سامانه

در پایان پروژه، سامانه قابلیت‌های زیر را به صورت یکپارچه در اختیار دارد:

- ثبت و مدیریت دستگاه‌های ESP32

- نگهداری نسخه فعلی و زمان آخرین حضور دستگاه

- بارگذاری و مدیریت نسخه‌های ثابت‌افزار

- محاسبه و بررسی SHA-256

- Update دستی و خودکار

- نصب متناوب روی دو پارتیشن OTA

- نگهداری وضعیت عملیات در NVS

- اعتبارسنجی بوت نسخه جدید

- Rollback خودکار

- Heartbeat و بررسی دوره‌ای نسخه جدید
- مدیریت اتصال مجدد WiFi
- تاریخچه مرحله‌ای Update
- مدیریت نسخه‌های فعال و غیرفعال
- جلوگیری از Job تکراری و Retry نسخه ناموفق

۱۹. جمع‌بندی

در این پروژه یک مسیر کامل مدیریت OTA برای ESP32 طراحی و پیاده‌سازی شد. فرآیند توسعه از راه‌اندازی ساده ارتباط بورد و سرور آغاز شد و به سامانه‌ای رسید که قادر است نسخه‌های ثابت‌افزار را مدیریت کند، فایل را از سرور دریافت و روی پارتیشن مقصد نصب نماید، صحت آن را با SHA-256 بررسی کند، نسخه جدید را در نخستین بوت اعتبارسنجی کند و در صورت شکست به نسخه سالم قبلی بازگردد.

در سمت سرور نیز مدیریت دستگاه، Firmware، Update Job و تاریخچه رویدادها تکمیل شد و حالت‌های دستی و خودکار در کنار Heartbeat و Polling دوره‌ای در اختیار سامانه قرار گرفتند. مشکلاتی که طی تست‌ها مشاهده شدند، از جمله حلقه تکرار پس از Stack Overflow، Rollback و گزارش‌های تکراری وضعیت، در مراحل مختلف توسعه شناسایی و اصلاح شدند.

نتایج تست‌های نهایی نشان دادند که مسیرهای اصلی موفقیت، شکست و بازیابی سامانه به شکل مورد انتظار عمل می‌کنند. در نتیجه، خروجی نهایی پروژه یک سامانه عملی و قابل آزمایش برای مدیریت از راه دور ثابت‌افزار ESP32 است که اجزای کلاینت، سرور، پایگاه داده و پنل مدیریتی را در یک معماری یکپارچه به هم متصل می‌کند.

۲۰. راهنمای استفاده و اجرای سامانه

این بخش برای استفاده عملی از سامانه و همچنین افزودن قابلیت OTA به یک برنامه دیگر روی ESP32 ارائه شده است. برای حفظ رفتار صحیح به روزرسانی، لازم نیست منطق برنامه کاربر داخل فایل های OTA نوشته شود؛ بهتر است ماژول های OTA بدون تغییر در کنار فایل های برنامه اصلی باقی بمانند و برنامه کاربر فقط راه اندازی آن ها را انجام دهد.

۲۰-۱. استفاده از OTA در کنار کد برنامه کاربر

در پروژه فعلی، تابع `app_manager_start()` مسئول آماده سازی وضعیت OTA، اتصال شبکه، بررسی عملیات قبلی و ایجاد Task مستقل سرویس است. این Task پس از راه اندازی به صورت مستقل فعال می ماند و `Heartbeat` بررسی دوره ای نسخه جدید، بازیابی ارتباط شبکه و ارسال مجدد وضعیت های نهایی را انجام می دهد.

اگر کاربر فایل `main.c` مخصوص برنامه خود را دارد، کافی است `app_manager_start()` را یک بار و پیش از شروع حلقه یا Task اصلی برنامه فراخوانی کند. نمونه ساده به صورت زیر است:

```
1 #include "app_manager.h"
2 #include "user_app.h"
3
4 void app_main(void)
5 {
6     app_manager_start();
7     user_app_start();
8 }
```

پس از بازگشت `app_manager_start()`، سرویس OTA در Task جداگانه `ota_service` ادامه پیدا می کند و برنامه کاربر می تواند کار عادی خود را انجام دهد. اگر برنامه کاربر دارای حلقه دائمی است، بهتر است آن حلقه در یک Task جدا اجرا شود یا در آن از `delay/yield` مناسب استفاده شود تا Task سرویس OTA از زمان پردازنده محروم نشود. همچنین نباید `app_manager_start()` چند بار فراخوانی شود.

۲۰-۲. فایل‌های لازم برای حفظ قابلیت OTA

برای اضافه کردن این قابلیت به یک پروژه دیگر، مطمئن‌ترین روش این است که فایل‌های زیر در همان component برنامه ESP-IDF باقی بمانند:

فایل	کاربرد
app_manager.c / app_manager.h	مدیریت راه‌اندازی، بازیابی وضعیت قبلی و ایجاد Task دوره‌ای OTA
ota_manager.c / ota_manager.h	دانلود فایل، نوشتن روی پارتیشن، بررسی SHA-256 و انتخاب پارتیشن بوت
ota_state.c / ota_state.h / ota_types.h	نگهداری وضعیت عملیات در NVS و تعریف وضعیت‌ها و اطلاعات Update
api_client.c / api_client.h / api_config.h	ارتباط HTTP/JSON با سرور و تعیین آدرس سرور
wifi.c / wifi.h / wifi_config.h	اتصال اولیه و بازیابی اتصال WiFi
ota_validator.c / ota_validator.h	اعتبارسنجی نسخه جدید در نخستین بوت و تصمیم برای تأیید یا Rollback
idf_component.yml	وابستگی cJSON مورد استفاده برای پردازش پاسخ‌های سرور
partitions.csv	تعریف factory.ota_0 و ota_1 و otadata برای نصب و بازگشت نسخه‌ها

اگر برنامه کاربر main.c مخصوص خود را دارد، نیازی نیست فایل main.c نمونه پروژه کپی شود؛ تنها باید Header مربوط به app_manager اضافه و تابع راه‌اندازی آن فراخوانی شود. در مقابل، حذف هر یک از ماژول‌های بالا بدون جایگزین کردن همان رابط‌ها می‌تواند بخشی از دانلود، گزارش وضعیت، بازیابی شبکه یا Rollback را از کار بیندازد.

۲۰-۳. تنظیمات ضروری پروژه مقصد

فایل `main/CMakeLists.txt` باید فایل‌های مازول OTA را در بخش `SRCS` ثبت کند و وابستگی‌های مورد نیاز شامل `esp_wifi`، `esp_event`، `esp_netif`، `nvs_flash`، `esp_http_client`، `app_update` و `mbedtls` حفظ شوند.

همچنین در پروژه مقصد باید تنظیمات اصلی نسخه فعلی حفظ شوند: حافظه `Flash` روی `4MB` تنظیم شود، جدول پارتیشن سفارشی `partitions.csv` انتخاب شود و قابلیت `Rollback` بوت‌لودر فعال باشد. در نسخه فعلی این موارد در `sdkconfig` با گزینه‌های اصلی زیر اعمال شده‌اند:

```
1 CONFIG_BOOTLOADER_APP_ROLLBACK_ENABLE=y
2 CONFIG_ESPTOOLPY_FLASHSIZE_4MB=y
3 CONFIG_PARTITION_TABLE_CUSTOM=y
4 CONFIG_PARTITION_TABLE_CUSTOM_FILENAME="partitions.csv"
```

نسخه برنامه نیز از `PROJECT_VER` در فایل اصلی `CMakeLists.txt` خوانده می‌شود. هنگام ساخت نسخه جدید، مقدار این متغیر باید تغییر کند و همان شماره نسخه هنگام بارگذاری فایل `BIN` در سرور ثبت شود.

قبل از اجرا، مقادیر `WIFI_SSID` و `WIFI_PASSWORD` در `wifi_config.h` و آدرس سرور در `api_config.h` تنظیم می‌شوند. آدرس سرور باید از شبکه `ESP32` قابل دسترسی باشد؛ بنابراین هنگام اجرای سرور روی رایانه، باید آدرس `IP` همان رایانه در شبکه محلی استفاده شود.

فایل `ota_validator.c` نیز باید در پروژه باقی بماند. در پیاده‌سازی فعلی اعتبارسنجی پایه با مقدار موفق انجام می‌شود؛ در یک کاربرد واقعی می‌توان داخل همین تابع آزمون‌های سلامت برنامه، سنسورها یا بخش‌های ضروری سیستم را اضافه کرد. در صورت بازگرداندن مقدار ناموفق، مکانیزم `Rollback` فعال خواهد شد.

۲۰-۴. راه‌اندازی سرور و پنل مدیریتی

برای اجرای سرور ابتدا وارد پوشه `Code/server` شده و محیط `Python` و وابستگی‌ها آماده می‌شوند:

```
1 cd Code/server
2 ./scripts/01_setup.sh
3 ./scripts/02_run.sh
```

سرور به صورت پیش‌فرض روی پورت `8000` اجرا می‌شود. پنل مدیریتی در مسیر زیر در دسترس است:

```
1 http://<SERVER_IP>:8000/panel/
```

مستندات خودکار `FastAPI` در مسیر `/docs` و مسیر بررسی سلامت سرویس در `/api/health` قرار دارند. پوشه `Code/web_panel` باید در کنار پوشه `server` باقی بماند، زیرا سرور پنل را از همین مسیر ارائه می‌کند.

۵-۲۰. ساخت و اجرای برنامه روی ESP32

در فایل `scripts/01_env.sh` مسیر نصب ESP-IDF و پورت سریال مورد بررسی و در صورت نیاز اصلاح می‌شوند. سپس از پوشه کلاینت دستورات زیر اجرا می‌شوند:

```
1 cd Code/esp32_ota_client
2 ./scripts/02_deploy.sh
3 ./scripts/03_run.sh
```

اسکرپت اول برنامه را `Build` و از طریق `USB` روی `Flash` می‌کند و اسکرپت دوم خروجی `Serial Monitor` را نمایش می‌دهد. پس از `Flash` اولیه، دستگاه خود را در سرور ثبت می‌کند و `Task` سرویس دوره‌ای ایجاد می‌شود. از این مرحله به بعد برای به‌روزرسانی عادی نیازی به `Flash` از طریق `USB` نیست.

برای ساخت یک نسخه جدید، ابتدا مقدار `PROJECT_VER` تغییر داده شده و پروژه `Build` می‌شود. فایل باینری تولیدشده از پوشه `build` در صفحه `Firmware` پنل بارگذاری می‌شود. شماره نسخه ثبت‌شده در پنل باید با نسخه داخل باینری یکسان باشد. سپس با توجه به حالت دستگاه، `Update` به صورت دستی ایجاد می‌شود یا در حالت `automatic` توسط سرور انتخاب خواهد شد.